potassco / clorm Public

Code Issues 19 Pull requests 1 Actions Projects Security Insights

master Go to file Code



daveraja Merge pull request #121 from potassco/fix_func ... on Mar 16 **755**

.github	Remove CI testing for Python 3.6	7 months ago
clorm	Increment version to 1.4.2	2 months ago
docs	format code with black	7 months ago
examples	cleanup imports	7 months ago
profiling	cleanup imports	7 months ago
tests	Fix the clorm func() function.	2 months ago
.gitignore	Added emacs backup files to gitignore	4 years ago
.pre-commit-config.yaml	add configuration for isort	7 months ago
.readthedocs.yaml	Added readthedocs configuration to fix missing typing…	2 years ago
LICENSE	Initial commit	5 years ago
MANIFEST.in	Updated setup.py and MANIFEST.in for clorm.orm dir…	2 years ago
README.rst	Update README.rst	2 years ago

About

🗃️ A Python ORM-like interface for the Clingo Answer Set Programming (ASP) reasoner

clorm.readthedocs.io

Readme

MIT license

34 stars

6 watching

4 forks

Report repository

Releases 20

Version 1.4.1 release Latest

on Sep 14, 2022

+ 19 releases

Packages

generate_overloads.py	format imports with isort	7 months ago
pyproject.toml	add configuration for isort	7 months ago
requirements-dev.txt	remove precommit version to hopefully run precommit…	7 months ago
requirements.txt	Rebasing	2 years ago
runexamples.sh	Fixed combine_fields example so that it returns a nor…	3 years ago
runtests.py	format code with black	7 months ago
setup.py	format imports with isort	7 months ago

README.rst

Clingo ORM (Clorm)

Clorm is a Python library that provides an Object Relational Mapping (ORM) interface to the Clingo Answer Set Programming (ASP) solver.

For background, ASP is a declarative language for describing, and solving, hard search problems. Clingo is a feature rich ASP solver with an extensive, but relatively low-level, Python API.

The goal of this library is to make it easier to integrate Clingo within a Python application. It is implemented on top of the official Clingo API so is designed to supplement and not replace the Clingo API.

No packages published

Used by 5



Contributors 5



Languages

- **Python** 99.8%
- **Other** 0.2%

When integrating an ASP program into an application you typically want to model the domain as a statically written ASP program, but then to generate problem instances and process the results dynamically. Clorm makes this integration cleaner, both in terms of code readability but also by providing a framework that makes it easier to refactor the python code as the ASP program evolves.

The documentation is available online here.

Note: Clorm works with Python 3.5+ and Clingo 5.4+

Installation

Clorm requires Python 3.7+ and Clingo 5.4+. It can be installed using either the pip or conda package managers.

pip packages can be downloaded from PyPI:

```
$ pip install clorm
```

The alternative to install Clorm is with Anaconda. Assuming you have already installed some variant of Anaconda, first you need to install Clingo:

```
$ conda install -c potassco clingo
```

Then install Clorm:

```
$ conda install -c potassco clorm
```

Quick Start

The following example highlights the basic features of Clorm. The ASP and Python parts of this example are located in the `examples` sub-directory in the git repository. The ASP program is `quickstart.lp` and the Python program is `quickstart.py`. A clingo callable version with embedded Python is also provided and can be run with:

```
$ clingo embedded_quickstart.lp
```

Imagine you are running a courier company and you have drivers and items that need to be delivered on a daily basis. An item is delivered during one of four time slots, and you want to assign a driver to deliver each item, while also ensuring that all items are assigned and drivers aren't double-booked for a time slot.

You also want to apply some optimisation criteria. Firstly, you want to minimise the number of drivers that you use (for example, because bringing a driver on for a day has some fixed cost). Secondly, you want to deliver items as early in the day as possible.

The above crieria can be encoded with the following simple ASP program:

```
time(1..4).

1 { assignment(I, D, T) : driver(D), time(T) } 1 :- item(I).
:- assignment(I1, D, T), assignment(I2, D, T), I1 != I2.

working_driver(D) :- assignment(_,D,_).

#minimize { 1@2,D : working_driver(D) }.
#minimize { T@1,D : assignment(_,D,T) }.
```

This above ASP program encodes the *problem domain* and can be used to solve the problem for arbitrary instances by combining it with a *problem instance* (i.e., some combination of drivers and items).

We now use a Python program to dynamically generate the problem instance and to process the generated solutions. Each solution will be an assignment of drivers to items for a time slot.

First the relevant libraries need to be imported.

```
from clorm import Predicate, ConstantField, IntegerField
from clorm.clingo import Control
```

Note: Importing from `clorm.clingo` instead of `clingo`.

> While it is possible to use Clorm with the raw clingo library, a wrapper library is provided to make the integration seemless. This wrapper (should) behave identically to the original module, except that it extends the functionality to offer integration with Clorm objects. It is also possible to monkey patch Clingo if this is your preferred approach (see the documentation).

The next step is to define a data model that maps the Clingo predicates to Python classes. Clorm introduces a number basic concepts for defining the data model: a `Predicate` class that must be sub-classed, and various *Field* classes that correspond to definitions of allowable *logical terms* that form the parameters of predicates.

Clorm provides three standard field classes, `ConstantField`, `StringField`, and `IntegerField`, that correspond to the standard *logic programming* data types of integer, constant, and string. These fields are sub-classed from `RawField`.

```
class Driver(Predicate):
    name=ConstantField

class Item(Predicate):
    name=ConstantField

class Assignment(Predicate):
    item=ConstantField
    driver=ConstantField
    time=IntegerField
```

The above code defines three classes to match the ASP program's input and output predicates.

`Driver` maps to the `driver/1` predicate, `Item` maps to `item/1`, and `Assignment` maps to `assignment/3` (note: the `/n` is a common logic programming notation for specifying the arity of a predicate or function). A predicate can contain zero or more fields.

The number of fields in the `Predicate` declaration must match the predicate arity and the order in which they are declared must also match the position of each term in the ASP predicate.

Having defined the data model we now show how to dynamically add a problem instance, solve the resulting ASP program, and print the solution.

First the Clingo `Control` object needs to be created and initialised, and the static problem domain encoding must be loaded.

```
ctrl = Control(unifier=[Driver, Item, Assignment])
ctrl.load("quickstart.lp")
```

The `clorm.clingo.Control` object controls how the ASP solver is run. When the solver runs it generates *models*. These models constitute the solutions to the problem. Facts within a model are encoded as `clingo.Symbol` objects. The `unifier` argument defines how these symbols are turned into Predicate instances.

For every symbol fact in the model, Clorm will successively attempt to *unify* (or match) the symbol against the Predicates in the unifier list. When a match is found the symbol is used to define an instance of the matching predicate. Any symbol that does not unify against any of the predicates is ignored.

Once the control object is created and the unifiers specified the static ASP program is loaded.

Next we generate a problem instance by generating a lists of `Driver` and `Item` objects. These items are added to a `clorm.FactBase` object.

The `clorm.FactBase` class provides a specialised set-like container for storing facts (i.e., predicate instances). It provides the standard set operations but also implements a querying mechanism for a more database-like interface.

```
from clorm import FactBase

drivers = [ Driver(name=n) for n in ["dave", "morri", "michael" ] ]
items = [ Item(name="item{}".format(i)) for i in range(1,6) ]
instance = FactBase(drivers + items)
```

The `Driver` and `Item` constructors use named parameters that match the declared field names. Note: while you can use positional arguments to initialise instances, doing so will potentially make the code harder to refactor. So in general you should avoid using positional arguments except for a few cases (eg., simple tuples where the order is unlikely to change).

These facts can now be added to the control object and the combined ASP program grounded.

```
ctrl.add_facts(instance)
ctrl.ground([("base",[])])
```

At this point the control object is ready to be run and generate solutions. There are a number of ways in which the ASP solver can be run (see the Clingo API documentation). For this example, we use a mode where a callback function is specified. This function will then be called each time a model is found.

```
solution=None
def on_model(model):
    nonlocal solution        # Note: use `nonlocal` keyword depending on scope
    solution = model.facts(atoms=True)

ctrl.solve(on_model=on_model)
if not solution:
    raise ValueError("No solution found")
```

The `on_model()` callback is triggered for every new model. Because of the ASP optimisation statements this callback can potentially be triggered multiple times before an optimal model is found. Also, note that if the problem is unsatisfiable then it will never be called and you should always check for this case.

The line `solution = model.facts(atoms=True)` extracts only instances of the predicates that were registered with the `unifier` parameter. As mentioned earlier, any facts that fail to unify are ignored. In this case it ignores the `working_driver/1` instances. The unified facts are stored and returned in a `clorm.FactBase` object.

The final step in this Python program involves querying the solution to print out the relevant parts. To do this we call the `FactBase.select()` member function that returns a suitable `Select` object.

```
from clorm import ph1_

query=solution.query(Assignment)\
              .where(Assignment.driver == ph1_)\
              .order_by(Assignment.time)
```

A Clorm query can be viewed as a simplified version of a traditional database query, and the function call syntax will be familiar to users of Python ORM's such as SQLAlchemy or Peewee.

Here we want to find `Assignment` instances that match the `driver` field to a special placeholder object `ph1_` and to return the results sorted by the assignment time. The value of the `ph1_` placeholder will be provided when the query is actually executed; separating specification from execution allows the query to be re-run multiple times with different values.

In particular, we now iterate over the list of drivers and execute the query for each driver and print the result.

```
for d in drivers:
    assignments = list(query.bind(d.name).all())
    if not assignments:
        print("Driver {} is not working today".format(d.name))
    else:
        print("Driver {} must deliver: ".format(d.name))
        for a in assignments:
            print("\t Item {} at time {}".format(a.item, a.time))
```

Calling `query.bind(d.name)` first creates a new query with the placeholder values assigned. Because `d.name` is the first parameter it matches against the placeholder `ph1_` in the query definition. Clorm has four predefined placeholders but more can be created using the `ph_` function.

Running this example produces the following results:

```
$ cd examples
$ python quickstart.py
Driver dave must deliver:
         Item item5 at time 1
         Item item4 at time 2
Driver morri must deliver:
         Item item1 at time 1
         Item item2 at time 2
         Item item3 at time 3
Driver michael is not working today
```

The above example shows some of the main features of Clorm and how to match the Python data model to the defined ASP predicates. For more details about how to use Clorm see the documentation.

Other Clorm Features

Beyond the basic features outlined above there are many other features of the Clorm library. These include:

- You can define new sub-classes of `RawField` for custom data conversions. For example, you can define a `DateField` that represents dates in clingo in a string YYYY-MM-DD format and then use it in a predicate definition.

```
from clorm import StringField          # StringField is a sub-class of RawField
import datetime

class DateField(StringField):          # DateField is a sub-class of StringFiel
    pytocl = lambda dt: dt.strftime("%Y-%m-%d")
    cltopy = lambda s: datetime.datetime.strptime(s,"%Y-%m-%d").date()

class Delivery(Predicate):
    item=ConstantField
    date=DateField

dd1=Delivery(item="item1", date=datetime.date(2019,14,5))    # Create delivery
```

```
% Corresponding ASP code
delivery(item1, "2019-04-05").
```

- Clorm supports predicate definitions with complex-terms; using either a `ComplexTerm` class (which is in fact an alias for Predicate) or Python tuples. Every defined complex term has an associated `RawField` sub-class that can be accessed as a `Field` property of the complex term class.

```
from clorm import ComplexTerm

class Event(ComplexTerm):
    date=DateField
    name=StringField

class Log(Predicate):
    event=Event.Field
    level=IntegerField
```

```
l1=Log(event=Event(date=datetime.date(2019,4,5),name="goto shops"),level=0)


% Corresponding ASP code
log(event("2019-04-05", "goto shops"), 0).
```

- Function definitions can be decorated with a data conversion signature to perform automatic type conversion for writing Python functions that can be called from an ASP program using the @-syntax.

  For example a function `add` can be decorated with an data conversion signature that accepts two input integers and expects an output integer.

```
@make_function_asp_callable(IntegerField, IntegerField, IntegerField)
def add(a,b): a+b


% Calling the add function from ASP
f(@add(5,6)).    % grounds to f(11).
```

- The data conversion signature can also be specified using Python 3.x function annotations. So for an equivalent specification of `add` above:

```
@make_function_asp_callable
def add(a : IntegerField, b : IntegerField) -> IntegerField: a+b
```

- Note, the Clingo API does already perform some automatic data conversions. However these conversions are ad-hoc, in the sense that it will automatically convert numbers and strings, but cannot deal with other types such as constants or more complex terms.

In contrast the Clorm mechanism of a data conversion signatures provide a more complete and transparent approach; it can deal with arbitrary conversions and all data conversions are clear since they are specified as part of the signature.

Development

- Python version: Clorm is actively developed using recent Python versions (3.8+)
- Clingo version: Clorm is typically tested with both Clingo version 5.4 and 5.5

Ideas for the Future

Here are some thoughts on how to extend the library.

- Add more examples to show how to use the Clorm.

- Build a library of resuable ASP integration components. I've started on this but am unsure how useful it would be. While there are some general concepts that you might consider encoding (e.g., date and time), however, how you actually want to encode them could be application specific. For example, encoding time down to the second or minute level is probably not what you want for a calendar scheduling application. In such a case a higher granularity, say 15 min blocks, is better.

  It could be that rather than a library of components, a set of example templates that could be copied and modified might be more useful.

- Add a debug library. There are two aspects to debugging: debugging your Python-ASP integration code, and debugging the ASP code itself. For the first case, I should at least go through Clorm to make sure that any generated exceptions have meaningful error messages.

> Debugging ASP code itself is trickier. It is often a painful process; when you mess up you often end up with an unsatisfiable problem, which doesn't tell you anything about what went wrong. You then end up commenting out constraints until it becomes satisfiable and you can look at the models being generated. My ideas are only vague at this stage. Maybe a tool that automatically weakens constraints until the problem becomes satisfiable. There are a few papers on debugging ASP so need to chase these up and see if there is something I can use.

Alternatives

I think an ORM interface provides a natural fit for getting data into and out of the Clingo solver. However, there will be other opinions on this. Also, data IO is only one aspect of how you might want to interact with the ASP solver.

So, here are some other projects for using Python and Clingo:

- PyASP
- Clyngor

License

This project is licensed under the terms of the MIT license.